

CS2 AI COACHING SYSTEM

Ultimate CS2 Coach

Parte 1B — I Sensi e lo Specialista

RAP Coach Model, ChronovisorScanner, GhostEngine, Sorgenti Dati, HLTV,
Steam, FACEIT

Autore: Renan Augusto Macena

Agosto 2026

Ultimate CS2 Coach — Parte 1B: I Sensi e lo Specialista

Argomenti: Modello RAP Coach (architettura 7 componenti: Percezione, Memoria LTC+Hopfield, Strategia, Pedagogia, Attribuzione Causale, Posizionamento, Comunicazione), ChronovisorScanner (rilevamento momenti critici multi-scala), GhostEngine (pipeline inferenza 4-tensori), e tutte le sorgenti dati esterne (Demo Parser, HLTV, Steam, FACEIT, TensorFactory, FAISS, Round Context).

Autore: Renan Augusto Macena

Questo documento è la continuazione della **Parte 1A — Il Cervello: Architettura Neurale e Addestramento**, che documenta i modelli di rete neurale (JEPA, VL-JEPA, AdvancedCoachNN), il sistema di addestramento a 3 livelli di maturità, l'Osservatorio di Introspezione del Coach, e i fondamenti architetturali del sistema (principio NO-WALLHACK, contratto 25-dim).

Indice

Parte 1B — I Sensi e lo Specialista (questo documento)

4. Sottosistema 2 — Modello RAP Coach ([backend/nn/experimental/rap_coach/](#))

- RAPCoachModel (Input Visivi Duali: per-timestep e statici)
- Livello di Percezione (ResNet a 3 flussi)
- Livello di Memoria (LTC + Hopfield)
- Livello Strategia (SuperpositionLayer + MoE)
- Livello Pedagogico (Value Critic + Attribuzione Causale)
- Modello Latente delle Abilità
- RAP Trainer (Funzione di Perdita Composita)
- ChronovisorScanner (Rilevamento Momenti Critici Multi-Scala)
- GhostEngine (Pipeline Inferenza 4-Tensori con PlayerKnowledge)

5. Sottosistema 1B — Sorgenti Dati ([backend/data_sources/](#))

- Demo Parser + Demo Format Adapter
- Event Registry (Schema CS2 Events)
- Trade Kill Detector
- Steam API + Steam Demo Finder

- Modulo HLTV (stat_fetcher, FlareSolvrr, Docker, rate limiting adattivo)
- FACEIT API + Integration
- TensorFactory — Fabbrica dei Tensori (Percezione Player-POV NO-WALLHACK)
- Indice Vettoriale FAISS (Ricerca Semantica ad Alta Velocità)
- Contesto dei Round (Griglia Temporale)

Parte 1A — Il Cervello: Core della Rete Neurale (JEPA, VL-JEPA, AdvancedCoachNN, SuperpositionLayer, EMA, CoachTrainingManager, TrainingOrchestrator, ModelFactory, NeuralRoleHead, MaturityObservatory)

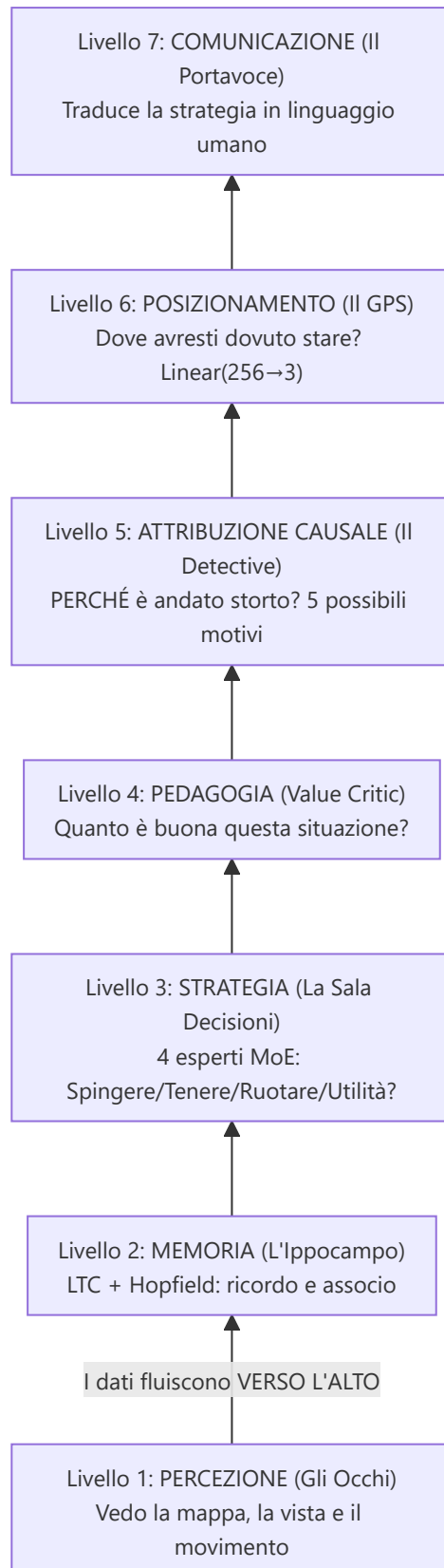
Parte 2 — Sezioni 5-13: Servizi di Coaching, Coaching Engines, Conoscenza e Recupero, Motori di Analisi (11), Elaborazione e Feature Engineering, Modulo di Controllo, Progresso e Tendenze, Database e Storage (Tri-Tier), Pipeline di Addestramento e Orchestrazione, Funzioni di Perdita

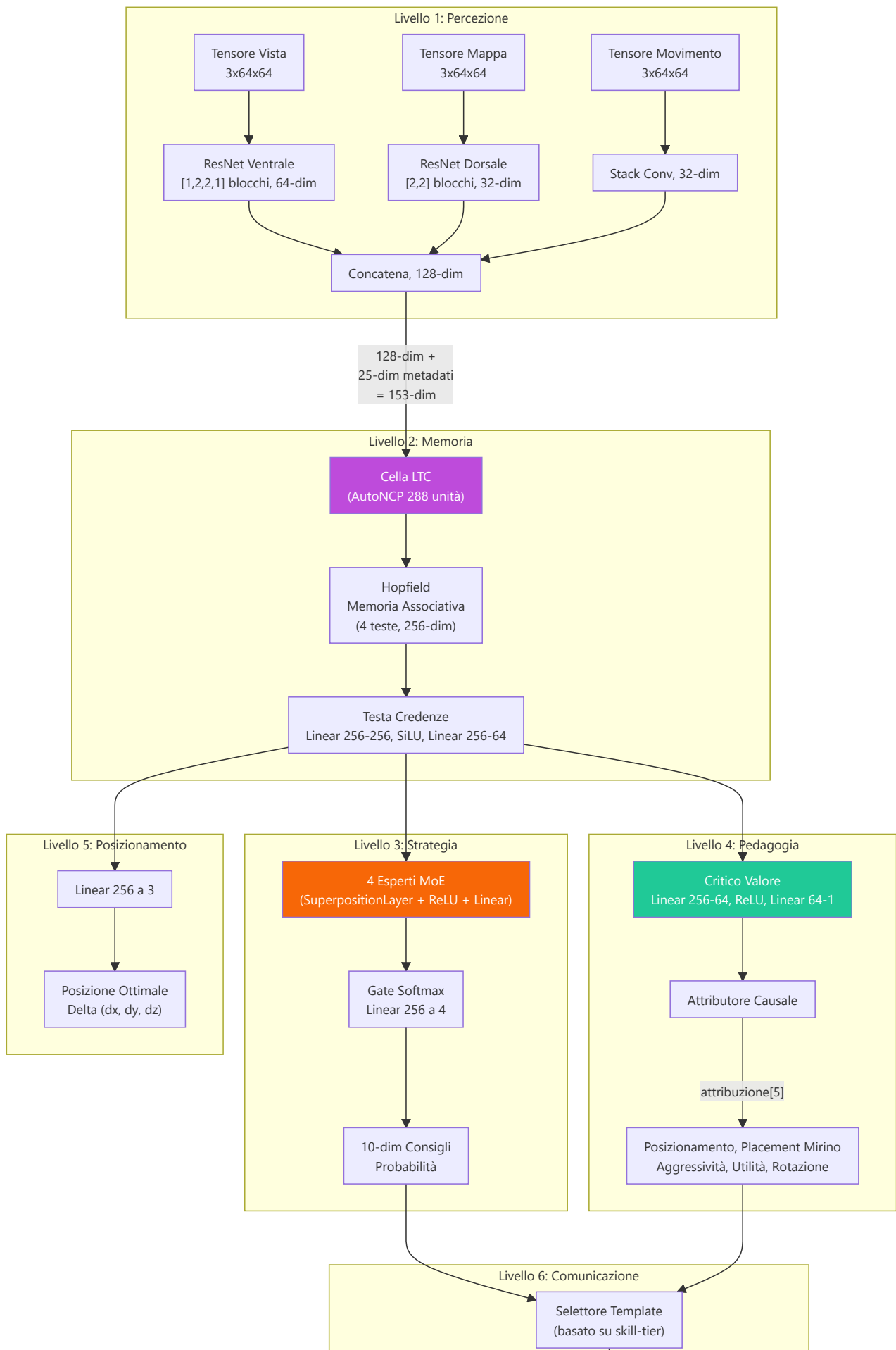
Parte 3 — Logica Programma, UI, Ingestion, Tools, Tests, Build, Remediation

4. Sottosistema 2 — Modello RAP Coach

Directory canonico: `backend/nn/experimental/rap_coach/` (il vecchio percorso `backend/nn/rap_coach/` è uno shim di reindirizzamento) **File:** `model.py`, `perception.py`, `memory.py`, `strategy.py`, `pedagogy.py`, `communication.py`, `trainer.py`, `chronovisor_scanner.py` (+ `test_arch.py` diagnostico standalone). Il modello latente delle abilità vive ormai in `backend/processing/skill_assessment.py` — il vecchio `rap_coach/skill_model.py` è uno shim di re-export.

Il RAP (Reasoning, Adaptation, Pedagogy) Coach è un'**architettura profonda con 6 componenti neurali apprendibili + 1 livello di comunicazione esterna**, appositamente progettata per il coaching CS2 in condizioni di osservabilità parziale (condizioni POMDP). La classe `RAPCoachModel` contiene Percezione (`RAPPerception`), Memoria (`RAPMemory` con LTC+Hopfield), Strategia (`RAPStrategy`), Pedagogia (`RAPPedagogy` con Value Critic e Skill Adapter), Attribuzione Causale (`CausalAttributor`) e una Testa di Posizionamento (`nn.Linear(256→3)`), tutti apprendibili. Il livello di Comunicazione (`communication.py`, classe `RAPCommunication`) opera esternamente come selettore di template di post-elaborazione — non è istanziato dentro il modello. Il forward pass produce 7 output: `advice_probs`, `belief_state`, `value_estimate`, `gate_weights`, `optimal_pos`, `attribution` e `hidden_state` (NN-40, stato ricorrente per inferenza continua).





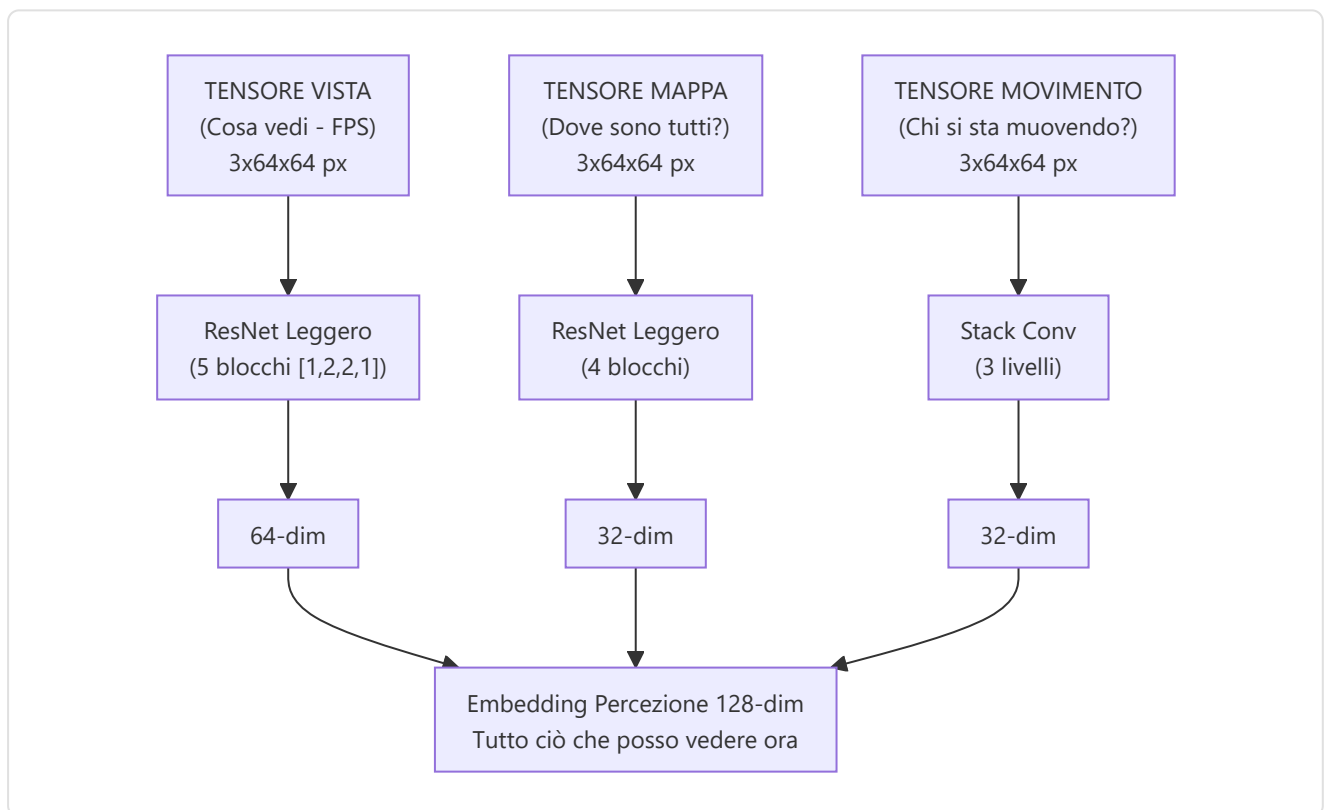
Stringa Consigli
Leggibile

-Livello di percezione (`perception.py`)

Un front-end **convoluzionale a tre flussi** che elabora gli input visivi:

Input	Forma	Backbone	Output Dim
Tensore di visualizzazione	3×64×64	Flusso ventrale ResNet: [1,2,2,1] blocchi, 3→64 canali	64-dim
Tensore di mappa	3×64×64	Flusso dorsale ResNet: [2,2] blocchi, 3→32 canali	32-dim
Tensore di movimento	3×64×64	Conv(3→16→32) + MaxPool + AdaptiveAvgPool	32-dim

I tre vettori di caratteristiche sono concatenati in un singolo **embedding di percezione a 128 dimensioni** (64 + 32 + 32).



I blocchi ResNet utilizzano **scorciatoie di identità** con downsample apprendibile (Conv1×1 + BatchNorm) quando stride ≠ 1 o il conteggio dei canali cambia. **24 livelli di convoluzione** su tutti e tre i flussi:

Flusso	Configurazione blocco	Blocchi	Conv/Blocco	Conversioni scorciatoie	Totale
Vista (Ventrale)	<code>[1,2,2,1]</code> → 1 + 5 = 6 blocchi	6	2	1 (primo blocco)	13
Mappa (Dorsale)	<code>[2,2]</code> → 1 + 3 = 4 blocchi	4	2	1 (primo blocco)	9
Movimento	Stack di conversione (2 livelli)	—	—	—	2
Totale					24

Come funziona `_make_resnet_stack` : Crea 1 blocco iniziale con `stride=2` (per il downsampling spaziale), quindi `sum(num_blocks) - 1` blocchi aggiuntivi con `stride=1`. Ogni `ResNetBlock` ha 2 livelli Conv2d (kernel 3×3). Il primo blocco riceve anche una scorciatoia Conv1×1 perché i canali di input (3) sono diversi dai canali di output (64 o 32).

Nota sulla scelta architettónica (F3-29): La configurazione originale `[3,4,6,3]` (15 blocchi, 33 conv nel flusso ventrale) era progettata per input 224×224 (la dimensione standard di ImageNet). Per input 64×64 come quelli utilizzati in questo progetto, le feature map collaserebbero spazialmente dopo il primo blocco stride-2, rendendo i blocchi successivi ridondanti. La configurazione `[1,2,2,1]` (5 blocchi effettivi) è calibrata specificamente per la risoluzione di training 64×64, con `AdaptiveAvgPool2d` che gestisce qualsiasi risoluzione spaziale residua. Eventuali checkpoint precedenti vengono automaticamente rilevati come `_stale_checkpoint` da `load_nn()`.

-Livello di memoria (`memory.py`) — LTC + Hopfield

Questa parte affronta la sfida fondamentale che il CS2 coach è un **Processo decisionale di Markov parzialmente osservabile** (POMDP).

Rete a costante di tempo liquida (LTC) con cablaggio AutoNCP:

- Input: 153 dim (128 percezione + 25 metadati)
- Unità NCP: **512** (`hidden_dim * 2` = 256 × 2), ripartite dal wiring AutoNCP in **154 neuroni motori / 143 command / 215 inter**; l'output motorio (154-dim) è proiettato a 256-dim tramite `ltc_projection = nn.Linear(154, 256)`
- Utilizza la libreria `ncps` con pattern di connettività sparsi, simili a quelli del cervello; un `_patched_ode_solver` interno corregge un bug di batch-broadcast di `ncps`
- Adatta la risoluzione temporale al ritmo del gioco (impostazioni lente vs. scontri a fuoco rapidi)
- Seeding deterministico (NN-MEM-02): numpy + torch RNG seedati a 42 durante la creazione del wiring AutoNCP, con ripristino dello stato RNG originale dopo l'inizializzazione — garantisce portabilità dei checkpoint tra diverse esecuzioni

Memoria associativa di Hopfield:

- Input/Output: 256-dim
- Teste: 4

- Utilizza `hflayers.HopfieldLayer` con **32 pattern-prototipo addestrabili** (`quantity=32` , $32 \times 256 = 8.192$ parametri) come **memoria indirizzabile tramite contenuto** per il recupero dei round prototipo
- I prototipi possono essere inizializzati via **K-means (K=32)** sui dati reali tramite `initialize_prototypes()` (Fase 5B)

Ritardo di attivazione Hopfield (NN-MEM-01 + RAP-M-04):

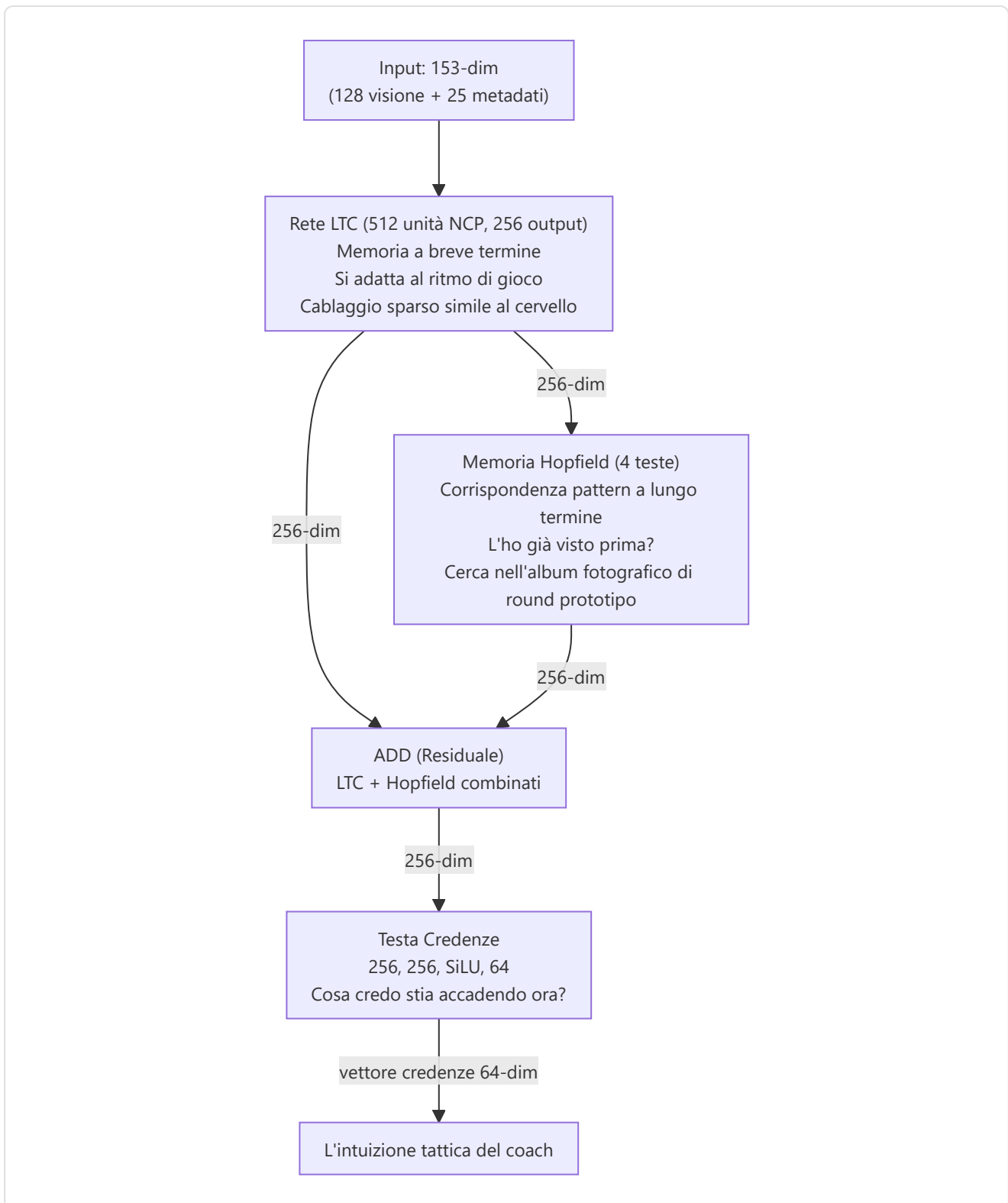
La rete di Hopfield **non si attiva immediatamente** durante l'addestramento: pattern non ancora modellati produrrebbero attenzione quasi uniforme su tutti gli slot, aggiungendo rumore anziché segnale. Per questo motivo:

- Il flag `_hopfield_trained` rimane `False` finché il trainer non chiama `notify_optimizer_step()` — cioè fino al **primo optimizer step reale** che ha modellato i pattern
- Prima dell'attivazione, il forward pass restituisce `torch.zeros_like(ltc_out)` al posto dell'output Hopfield
- Il caricamento di un checkpoint (`load_state_dict`) imposta `_hopfield_trained = True` immediatamente, assumendo che il modello sia già stato addestrato

RAPMemoryLite — Fallback LSTM puro:

Modulo sostitutivo leggero per `RAPMemory` , utilizzato quando le dipendenze `ncps` / `hflayers` non sono disponibili o quando si desidera un modello più portatile:

- LSTM standard PyTorch: `nn.LSTM(153, 256, batch_first=True)`
- Stesso contratto I/O: Input `[B, T, 153]` → Output (`combined_state [B, T, 256]`, `belief [B, T, 64]`, `hidden`)
- Stessa testa di credenze: `Linear(256→256) → SiLU → Linear(256→64)`
- Nessun seeding RNG necessario (niente AutoNCP)
- Nessun Hopfield training delay (niente pattern memorizzati)
- Istanziato via `ModelFactory.TYPE_RAP_LITE` ("rap-lite") con `use_lite_memory=True`



Combinazione residua: `combined_state = ltc_out + hopfield_out`

Testo di convinzione: `Lineare(256→256) → SiLU → Lineare(256→64)` — produce un vettore di convinzione a 64 dimensioni che codifica la comprensione tattica latente dell'allenatore.

Passaggio in avanti:

```

ltc_out, hidden = self.ltc(x, hidden) # x: [B, seq, 153] → [B, seq, 256]
mem_out = self.hopfield(ltc_out) # [B, seq, 256]
combined_state = ltc_out + mem_out # Residuo
belief = self.belief_head(combined_state) # [B, seq, 64]
return combined_state, belief, hidden

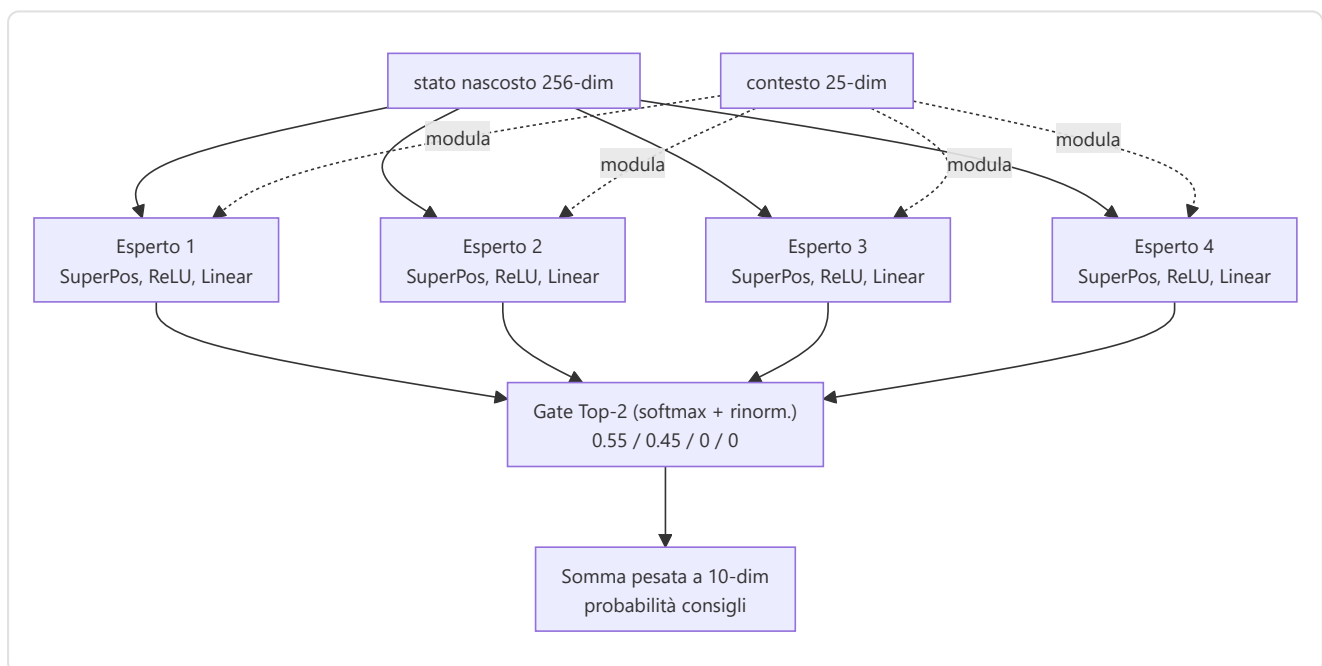
```

-Livello Strategia (`strategy.py`) — Sovrapposizione + MoE

Implementa **SuperpositionLayer** combinato con un mix di esperti contestualizzati a **routing sparso Top-2**:

SuperpositionLayer (`layers/superposition.py`): condizionamento **FiLM** dove $\text{output} = \gamma(\text{context}) \cdot (W \cdot x + b) + \beta(\text{context})$, con $\gamma = \text{sigmoid}(\text{context_gate}(\text{context}))$ e $\beta = \text{context_beta}(\text{context})$ inizializzato a zero (all'inizio il layer è puro gating moltiplicativo). Il peso di sparsità `context_gate_l1_weight = 1e-4` (da `HeuristicConfig`) scala la loss di sparsità del modello. Osservabile: le statistiche del gate (media, std, sparsità, active_ratio) possono essere tracciate.

Nota: `RAPStrategy.__init__` utilizza `context_dim=89` — il contesto è la concatenazione di **metadati (25) + vettore credenze (64)**. La rete di gate è `nn.Linear(hidden_dim=256, num_experts=4)` a logits grezzi: softmax → selezione **Top-2** (`TOP_K=2`) → rinormalizzazione dei due pesi selezionati, zero per gli altri due esperti.



4 Moduli Esperti: Ogni esperto è composto da `SuperpositionLayer (FiLM, context_dim=89) → ReLU → Linear(→10)`.

Gate Network: `nn.Linear(256→4)` (logits grezzi) → softmax → **Top-2 rinormalizzato** (gli altri due esperti restano a peso zero).

Output: Distribuzione di probabilità di consulenza a 10 dimensioni e vettore dei pesi di gate a 4 dimensioni (`gate_probs`).

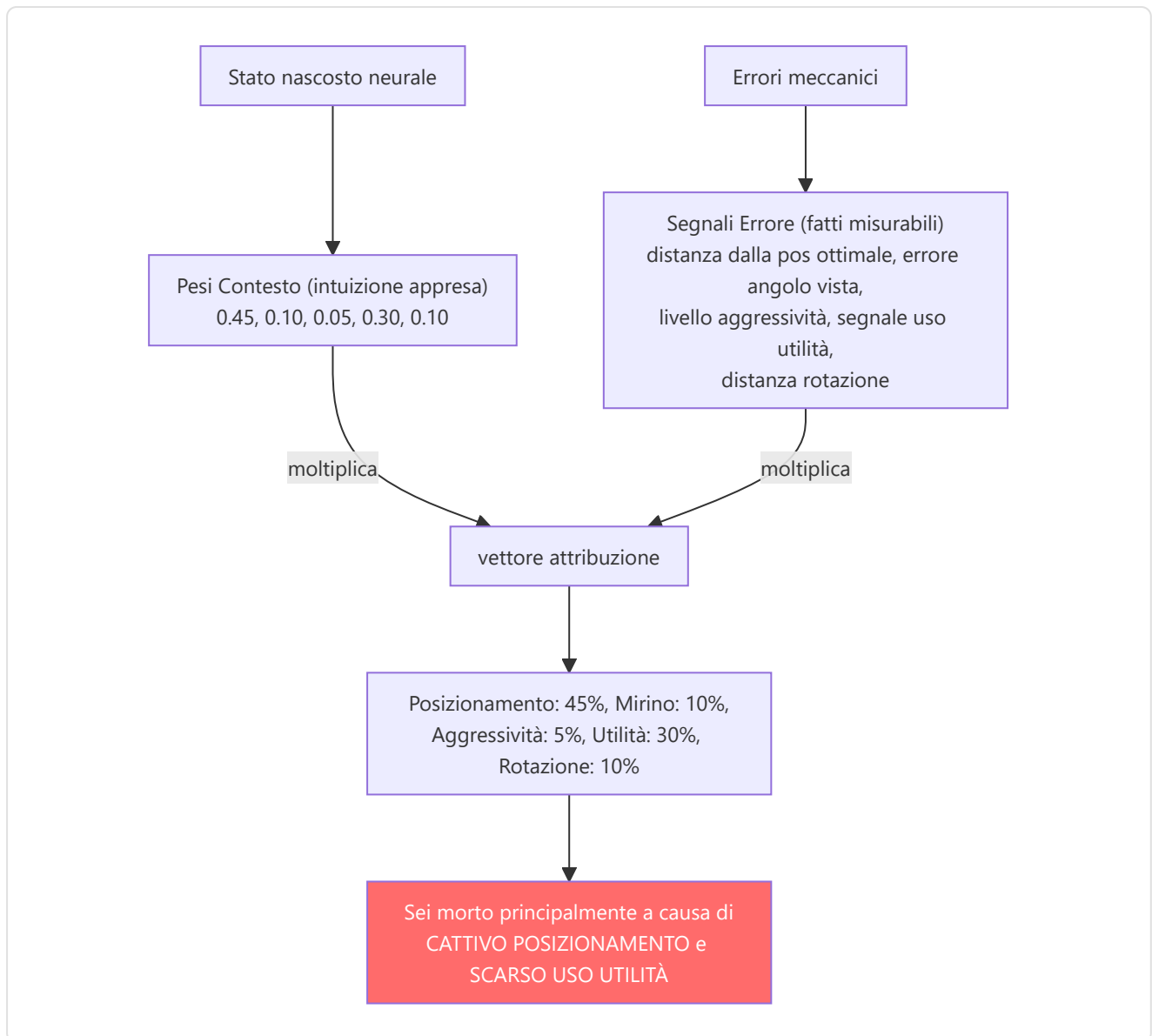
-Livello Pedagogico (`pedagogy.py`) — Valore + Attribuzione

Due sottomoduli:

1. **Value Critic:** `Linear(256→64) → ReLU → Linear(64→1)` . Stima $V(s)$ per l'apprendimento con differenze temporali. **Skill Adapter:** `Linear(10 skill_buckets → 256)` consente stime di valore condizionate dalle abilità.
2. **CausalAttributor:** Produce un vettore di attribuzione a 5 dimensioni che mappa i concetti di allenamento:

Indice	Concetto	Segnale meccanico
0	Posizionamento	<code>norm(position_delta)</code>
1	Posizionamento del mirino	<code>norm(view_delta)</code>
2	Aggressione	<code>0,5 × position_delta</code>
3	Utilità	<code>sigmoid(hidden.mean())</code> — segnale appreso e dipendente dal contesto : produce un'attivazione alta quando la rete rileva situazioni dove l'uso dell'utilità era rilevante, bassa quando il contesto tattico rende l'utilità secondaria. Non è un placeholder statico, ma una funzione non-lineare dello stato nascosto che si adatta durante l'addestramento
4	Rotazione	<code>0,8 × position_delta</code>

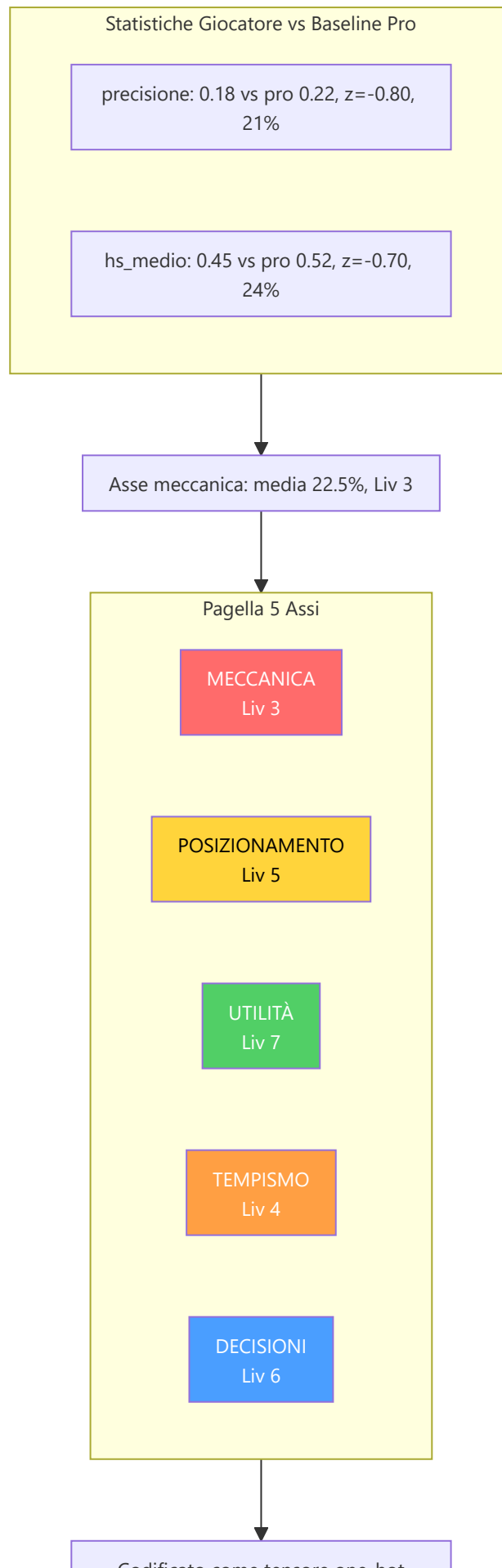
Fusione: `attribuzione = context_weights × mechanical_errors` dove `context_weights` deriva da `Lineare(256→32) → ReLU → Lineare(32→5) → Sigmoid` .



-Modello latente delle abilità (`skill_model.py`)

Scompone le statistiche grezze in 5 assi delle abilità utilizzando la normalizzazione statistica rispetto alle linee di base dei professionisti:

Asse delle abilità	Statistiche di input	Normalizzazione
Meccaniche	Precisione, avg_hs	Punteggio Z (μ =pro_mean, σ =pro_std)
Posizionamento	Valutazione_sopravvivenza, valutazione_kast	Punteggio Z
Utilità	Utility_blind_time, Utility_nemici_accecati	Punteggio Z
Tempistica	Percentuale_vittorie_duello_apertura, Punteggio_aggressione_posizionale	Punteggio Z
Decisione	Percentuale_vittorie_clutch, Impatto_valutazione	Punteggio Z



Codificato come tensore one-hot
Alimentato all'Adattatore Skill del
Livello Pedagogia

I punteggi Z vengono convertiti in percentili tramite l'**approssimazione logistica** $1/(1+\exp(-1,702z))$ (approssimazione CDF rapida), quindi il percentile medio viene mappato a un **livello curriculare** (1–10) tramite `int(avg_skill * 9) + 1`, fissato a [1, 10]. Il livello viene codificato come un tensore one-hot (10-dim) tramite `SkillLatentModel.get_skill_tensor()` per l'adattatore di competenze del livello pedagogico.

-RAP Trainer (`trainer.py`)

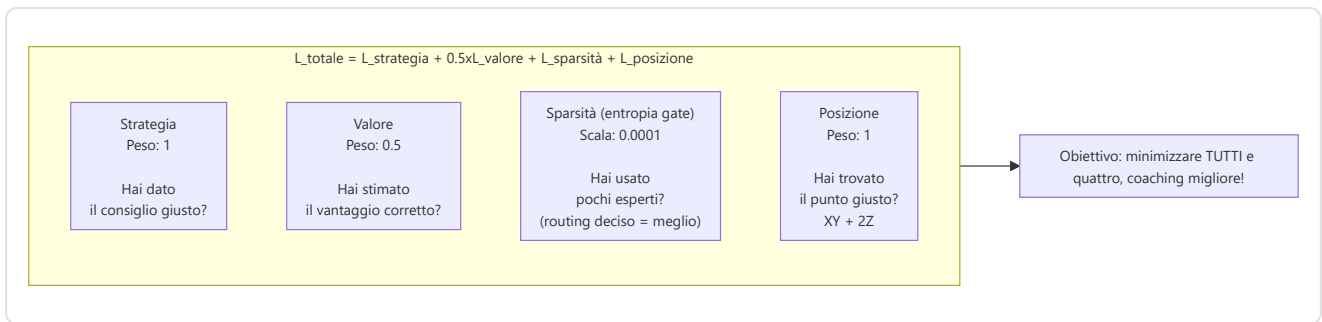
Orchestra il ciclo di addestramento con una **funzione di perdita composita**:

```
L_totale = 1.0 × L_strategia + 0.5 × L_valore + 1.0 × L_sparsità + 1.0 × L_posizione
```

Termine di perdita	Formula	Peso	Scopo
<code>L_strategy</code>	<code>MSELoss(advice_probs, target_strat)</code>	1.0	Raccomandazione tattica corretta
<code>L_value</code>	<code>MSELoss(V(s), true_advantage)</code> (con <code>val_mask</code> opzionale)	0.5	Stima accurata del vantaggio
<code>L_sparsity</code>	<code>model.compute_sparsity_loss(gate_weights)</code> — entropia del gate : $\text{context_gate_l1_weight} \times (-\sum p \cdot \log p)$, range [0, $\log 4 \approx 1.386$], scala 1e-4 (parametro esplicito, thread-safe F3-07)	1.0	Specializzazione esperta (bassa entropia = routing deciso)
<code>L_position</code>	<code>MSE(pred_x) + MSE(pred_y) + 2.0 × MSE(pred_z)</code>	1.0	Posizionamento ottimale, penalità rigorosa sull'asse Z

Ottimizzazione: AdamW (lr=1e-4, weight_decay=1e-2), scheduler `CosineAnnealingLR(T_max=100, eta_min=1e-6)`, **accumulo gradienti** su 4 step (`accumulation_steps=4`), AMP `GradScaler` (solo CUDA), `clip_grad_norm_(max_norm=1.0)`. Dopo ogni optimizer step reale, `_notify_memory_step()` notifica la memoria — è questo che attiva l'Hopfield (NN-MEM-01).

Nota: Il moltiplicatore 2× sull'asse Z esiste perché gli errori di posizionamento verticale (ad esempio, un livello sbagliato su Nuke/Vertigo) sono tatticamente catastrofici: rappresentano errori di piano sbagliato che nessuna correzione orizzontale può correggere.



Output per fase di addestramento: `{loss, sparsity_ratio, loss_pos, z_error}`.

-Riepilogo del passaggio in avanti di RAPCoachModel

Firma: `forward(view_frame, map_frame, motion_diff, metadata, skill_vec=None, hidden_state=None)` — il parametro `hidden_state` (NN-40) permette di passare lo stato ricorrente della memoria da una chiamata all'altra, abilitando **inferenza continua** senza cold-start: il GhostEngine può mantenere la memoria tra tick consecutivi anziché ripartire da zero ad ogni valutazione.

Fix NN-39 — Input Visivi Duali: Il passaggio in avanti gestisce due formati di input visivo attraverso un controllo dimensionale esplicito:

Formato Input	Shape	Quando si usa	Comportamento
Per-timestep	<code>[B, T, C, H, W]</code> (5-dim)	Addestramento con sequenze temporali	Ogni timestep elaborato individualmente dalla CNN
Statico	<code>[B, C, H, W]</code> (4-dim)	Inferenza in tempo reale (GhostEngine)	Singolo frame espanso su tutti i timestep

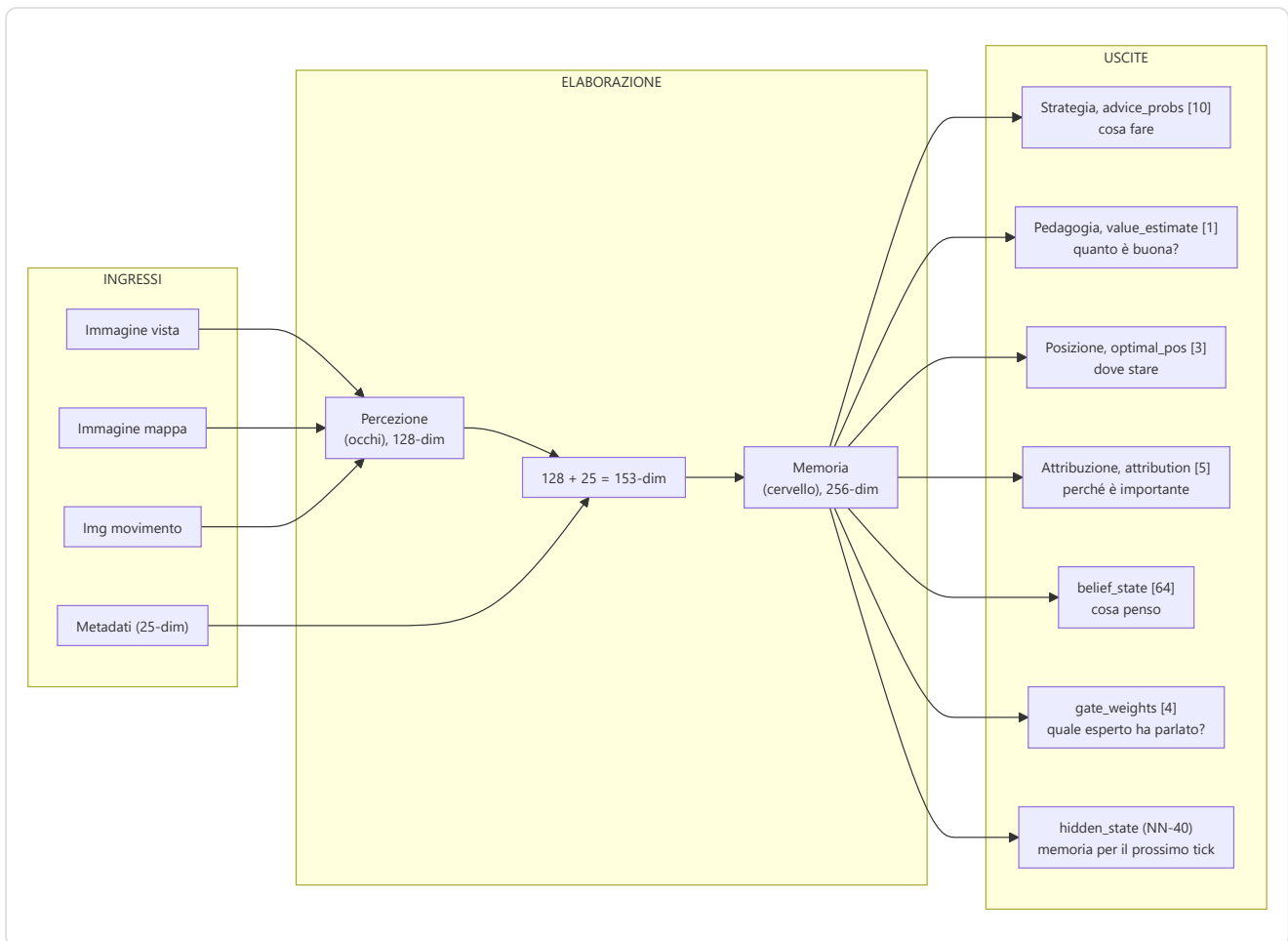
```

def forward(view_frame, map_frame, motion_diff, metadata, skill_vec=None):
    batch_size, seq_len, _ = metadata.shape

    # NN-39 fix: supporta input visivo per-timestep [B,T,C,H,W] e statico [B,C,H,W]
    if view_frame.dim() == 5:
        # Per-timestep - elabora ogni timestep attraverso la CNN separatamente
        z_frames = []
        for t in range(view_frame.shape[1]):
            z_t = self.perception(view_frame[:, t], map_frame[:, t], motion_diff[:, t])
            z_frames.append(z_t)
        z_spatial_seq = torch.stack(z_frames, dim=1)      # [B, T, 128]
    else:
        # Statico - singolo frame espanso su tutti i timestep
        z_spatial = self.perception(view_frame, map_frame, motion_diff) # [B, 128]
        z_spatial_seq = z_spatial.unsqueeze(1).expand(-1, seq_len, -1) # [B, T, 128]

    lstm_in = cat([z_spatial_seq, metadata], dim=2)      # [B, seq, 153]
    hidden_seq, belief, new_hidden = self.memory(lstm_in, hidden=hidden_state) # [B, seq, 256],
    last_hidden = hidden_seq[:, -1, :]
    prediction, gate_weights = self.strategy(last_hidden, context) # [B, 10], [B, 4]
    value_v = self.pedagogy(last_hidden, skill_vec)      # [B, 1]
    optimal_pos = self.position_head(last_hidden)        # [B, 3]
    attribution = self.attributor.diagnose(last_hidden, optimal_pos) # [B, 5]
    return {
        "advice_probs": prediction,      # [B, 10]
        "belief_state": belief,          # [B, seq, 64]
        "value_estimate": value_v,       # [B, 1]
        "gate_weights": gate_weights,    # [B, 4]
        "optimal_pos": optimal_pos,      # [B, 3]
        "attribution": attribution,      # [B, 5]
        "hidden_state": new_hidden,      # NN-40: stato ricorrente per inferenza continua
    }

```



-ChronovisorScanner ([chronovisor_scanner.py](#))

Un **modulo di elaborazione del segnale multi-scala** che identifica i momenti critici nelle partite analizzando i delta di vantaggio temporale su **3 livelli di risoluzione** (micro, standard, macro):

Configurazione Multi-Scala ([ANALYSIS_SCALES](#)):

Scala	Window (tick)	Lag	Soglia	Descrizione
Micro	64	16	0.10	Decisioni di ingaggio sotto-secondo
Standard	192	64	0.15	Momenti critici a livello di ingaggio
Macro	640	128	0.20	Rilevamento cambiamenti strategici (5-10 secondi)

Pipeline di rilevamento (per ciascuna scala):

1. Utilizza il modello RAP addestrato per prevedere $V(s)$ per ogni tick window.
2. Calcola i delta utilizzando il lag configurato per la scala: `deltas = values[LAG:] - values[:-LAG]`.
3. Rileva i **picchi** in cui `|delta| > soglia` (variabile per scala: 0.10/0.15/0.20).
4. Cerca il picco all'interno della finestra configurata, mantenendo la coerenza del segno.
5. La **suppressione non massima** impedisce rilevamenti duplicati.

- Classifica ogni picco come **"gioco"** (gradiente positivo, vantaggio acquisito) o **"errore"** (negativo, vantaggio perso).
- Restituisce istanze della classe di dati `CriticalMoment` con `(match_id, start_tick, peak_tick, end_tick, severity [0-1], type, description, scale)`.

Limite di sicurezza tick (F3-21): `_MAX_TICKS_PER_SCAN = 50.000` — le partite con più di 50K tick (possibile con overtime estesi o match molto lunghi) vengono **troncate** con un warning (NN-CV-02) anziché saturare la RAM. Il sistema preleva `_MAX_TICKS_PER_SCAN + 1` tick per rilevare la troncatura e avvisa che i momenti critici della fase finale della partita potrebbero essere persi.

Deduplicazione cross-scala: Quando lo stesso momento viene rilevato a scale diverse (es. un peek critico visibile sia nella scala micro che standard), la deduplicazione assegna priorità **micro > standard > macro** (la scala più fine vince). `MIN_GAP_TICKS = 64` (~1 secondo) definisce la distanza minima tra due momenti: se due picchi sono più vicini di 64 tick, vengono considerati lo stesso evento e viene mantenuto solo quello a scala più fine.

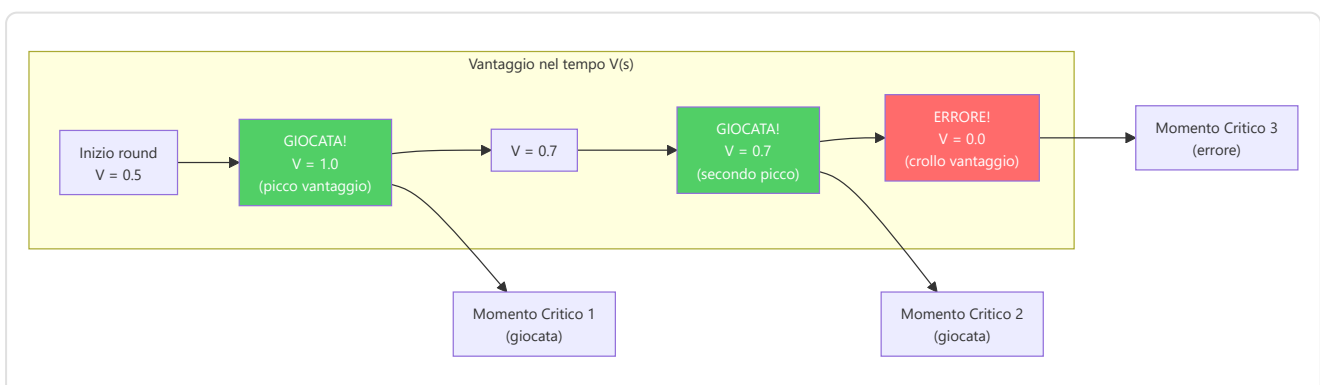
Etichette di severità: La severità (0-1) viene classificata automaticamente per il `MatchVisualizer`:

- `severity > 0.3` → **"critical"** (momento che cambia la partita)
- `severity > 0.15` → **"significant"** (momento rilevante)
- altrimenti → **"notable"** (momento degno di nota)

ScanResult dataclass: Tipo di ritorno strutturato che distingue successo da fallimento:

Campo	Tipo	Descrizione
<code>critical_moments</code>	<code>List[CriticalMoment]</code>	Momenti critici rilevati
<code>success</code>	<code>bool</code>	True se la scansione è completata (anche con 0 momenti)
<code>error_message</code>	<code>Optional[str]</code>	Dettaglio errore se <code>success=False</code>
<code>model_loaded</code>	<code>bool</code>	Se il modello RAP era disponibile
<code>ticks_analyzed</code>	<code>int</code>	Numero di tick effettivamente analizzati

Proprietà di utilità: `is_empty_success` (scansione riuscita ma nessun momento critico trovato), `is_failure` (scansione fallita — modello non caricato, errore DB, ecc.).



-GhostEngine (`inference/ghost_engine.py`)

Inferenza in tempo reale per il "Ghost" — overlay della posizione ottimale del giocatore. Il GhostEngine rappresenta il **punto finale** dell'intera catena neurale: è dove il RAP Coach Model produce output visibili all'utente sotto forma di un "giocatore fantasma" sulla mappa tattica.

Pipeline di Inferenza 4-Tensori con PlayerKnowledge:

La pipeline di inferenza opera in 5 fasi sequenziali per ogni tick di riproduzione:

Fase 1 — Caricamento Modello (`_load_brain()`)

- Verifica `USE_RAP_MODEL` da configurazione (interruttore generale)
- `ModelFactory.get_model(ModelFactory.TYPE_RAP)` — istanzia il modello RAP
- `load_nn(checkpoint_name, model)` — carica i pesi dal checkpoint su disco
- `model.to(device) → model.eval()` — sposta su GPU/CPU e attiva modalità inferenza
- In caso di fallimento: `model = None`, `is_trained = False` — disabilita previsioni

Fase 2 — Costruzione Tensori di Input

Tensore	Metodo	Shape Output	Contenuto
Map	<code>tensor_factory.generate_map_tensor(ticks, map_name, knowledge)</code>	<code>[1, 3, 64, 64]</code>	Posizioni compagni, nemici visibili, utilità + bomba
View	<code>tensor_factory.generate_view_tensor(ticks, map_name, knowledge)</code>	<code>[1, 3, 64, 64]</code>	Maschera FOV 90°, entità visibili, zone utilità
Motion	<code>tensor_factory.generate_motion_tensor(ticks, map_name)</code>	<code>[1, 3, 64, 64]</code>	Traiettoria 32 tick, campo velocità, delta mirino
Metadata	<code>FeatureExtractor.extract(tick_data, map_name, context)</code>	<code>[1, 1, 25]</code>	Vettore canonico 25-dim (salute, posizione, economia, ecc.)

Il **ponte PlayerKnowledge** (`_build_knowledge_from_game_state()`) filtra i dati secondo il principio NO-WALLHACK: solo le informazioni legittimamente disponibili al giocatore (compagni, nemici visibili, ultime posizioni note con decadimento) vengono codificate nei tensori mappa e vista. Se la costruzione della conoscenza fallisce, il sistema degrada alla modalità legacy (tensori vuoti).

Fase 2b — Modalità POV (R4-04-01):

Modalità	Condizione	Comportamento
POV Mode	<code>USE_POV_TENSORS=True</code> + <code>game_state</code> fornito	Costruisce <code>PlayerKnowledge</code> dal game state → tensori POV con semantica canale dedicata
Legacy Mode	<code>USE_POV_TENSORS=False</code> (default)	Tensori standard allineati con i dati di addestramento

Attenzione (R4-04-01): I tensori POV utilizzano una semantica dei canali diversa (Ch0=compagni, Ch1=nemici ultimi noti) rispetto ai dati di addestramento standard (Ch0=nemici, Ch1=compagni). Usare tensori POV con un modello addestrato in modalità legacy produrrà risultati **inaffidabili**. La modalità POV è valida solo se il modello è stato addestrato con dati POV.

Fase 3 — Inferenza Neurale

```
with torch.no_grad():
    out = self.model(view_frame=view_t, map_frame=map_t,
                     motion_diff=motion_t, metadata=meta_t,
                     hidden_state=self._last_hidden) # NN-40: stato persistente
    self._last_hidden = out["hidden_state"] # Mantieni per il prossimo tick
```

`torch.no_grad()` disabilita il calcolo dei gradienti (solo inferenza, nessun addestramento). Il parametro `hidden_state` (NN-40) permette di mantenere lo stato ricorrente della memoria tra tick consecutivi, evitando il cold-start ad ogni valutazione.

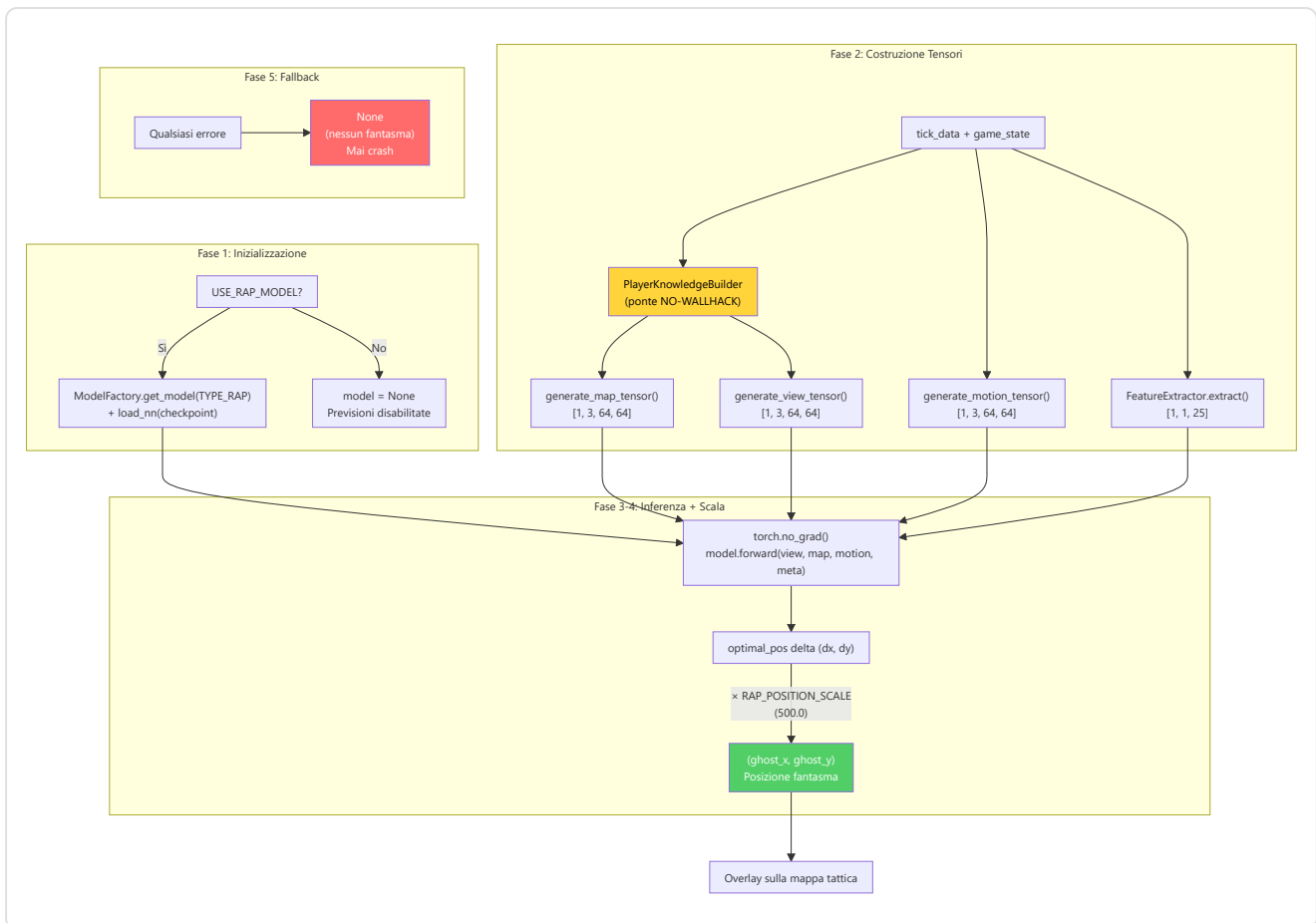
Fase 4 — Decodifica e Scala Posizione

```
optimal_delta = out["optimal_pos"].cpu().numpy()[0] # [dx, dy, dz]
ghost_x = current_x + (optimal_delta[0] * RAP_POSITION_SCALE) # × 500.0
ghost_y = current_y + (optimal_delta[1] * RAP_POSITION_SCALE) # × 500.0
return (ghost_x, ghost_y)
```

Il modello produce un delta normalizzato in $[-1, 1]$ che viene scalato a coordinate mondo tramite `RAP_POSITION_SCALE = 500.0` (da `config.py`). La costante è condivisa tra GhostEngine e overlay per garantire coerenza.

Fase 5 — Fallback Graduale (5 modalità)

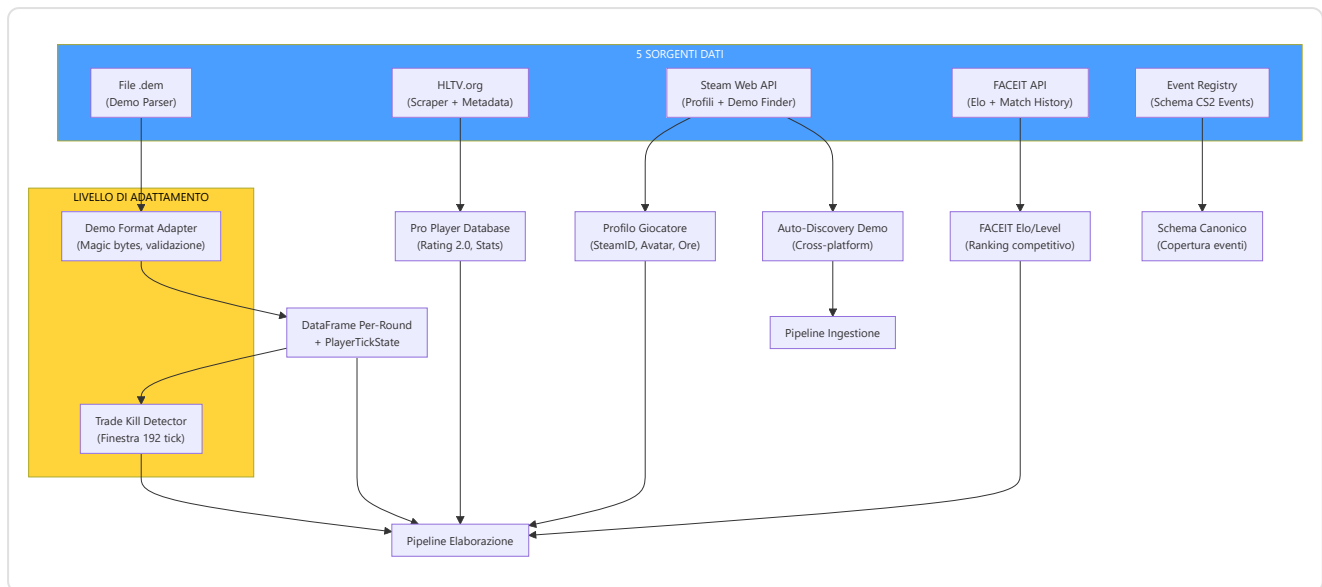
Modalità Fallback	Condizione	Comportamento
Modello disabilitato	<code>USE_RAP_MODEL=False</code> (default)	Skip caricamento, nessuna predizione (<code>None</code>)
Checkpoint mancante	Addestramento non completato	<code>model = None</code> , <code>is_trained = False</code> , previsioni disabilitate
Nome mappa mancante	Nessun contesto spaziale	Ritorna <code>None</code> immediatamente
Errore PlayerKnowledge	Costruzione conoscenza fallita	Degrada a tensori legacy
Errore di inferenza	RuntimeError / CUDA OOM	Log errore, ritorna <code>None</code>



5. Sottosistema 1B — Sorgenti Dati

Cartella nel programma: `backend/data_sources/` **File:** `demo_parser.py`, `demo_format_adapter.py`, `event_registry.py`, `trade_kill_detector.py`, `round_context.py`, `hltv_scraper.py`, `steam_api.py`, `steam_demo_finder.py`, `faceit_api.py`, `faceit_integration.py`, `__init__.py` + sottopacchetto `hltv/` (`stat_fetcher.py`, `flaresolvr_client.py`, `docker_manager.py`)

Il sottosistema Sorgenti Dati è il **punto di ingresso di tutti i dati esterni** nel sistema. Raccoglie informazioni da 5 fonti distinte: file demo CS2, statistiche HLTV, profili Steam, dati FACEIT e registry eventi di gioco.



-Demo Parser (`demo_parser.py`)

Wrapper robusto attorno alla libreria `demoparser2` per l'estrazione di statistiche da file demo CS2. Il rating è calcolato tramite il **modulo di valutazione unificato** `feature_engineering/rating.py`, che definisce le baseline HLTV 2.0:

Costante	Valore	Significato
<code>BASELINE_KPR</code>	0.679	Media pro: uccisioni per round
<code>BASELINE_DPR_COMPLEMENT</code>	0.317	Media pro: tasso di sopravvivenza
<code>BASELINE_KAST</code>	0.70	Media pro: Kill/Assist/Survive/Trade %
<code>BASELINE_IMPACT</code>	1.0	Media pro: impact rating
<code>BASELINE_ADR</code>	73.3	Media pro: danno medio per round

`parse_demo(demo_path, target_player=None)`: Entry point principale. Validazione di esistenza file, parsing eventi `round_end` per contare i round, poi estrazione statistica completa via `_extract_stats_with_full_fields()`. Restituisce `pd.DataFrame` vuoto in caso di qualsiasi errore (fail-safe).

`_extract_stats_with_full_fields(parser, total_rounds, target_player)`: Calcola tutte le 25 feature aggregate obbligatorie per il database:

- Statistiche base: `avg_kills`, `avg_deaths`, `avg_adr`, `kd_ratio`
- Varianza: `kill_std`, `adr_std` (via `_compute_per_round_variance`)
- Statistiche avanzate: `avg_hs`, `accuracy`, `impact_rounds`, `econ_rating`
- Rating HLTV 2.0 approssimato (approssimazione hand-tuned, non formula ufficiale)

La guardia di parsing (`parse_guard.py`). `demoparser2` è un'estensione Rust, e una demo malformata non solleva un'eccezione Python normale: fa **andare in panico il codice Rust**, che `pyo3` traduce in una `PanicException`. Quella classe deriva da `BaseException`, non da `Exception` — quindi ogni `except`

Exception della pipeline la lasciava passare, e una singola demo corrotta interrompeva l'intera sessione di ingestione, nonostante i docstring promettessero il contrario.

La correzione non poteva essere `except pyo3_runtime.PanicException`, perché **pyo3 crea quella classe pigramente, al primo panic**: prima di allora il modulo non è nemmeno importabile. La guardia riconosce quindi l'eccezione **per nome di classe**:

```
_PROPAGATE = (KeyboardInterrupt, SystemExit, GeneratorExit)

def is_parse_error(exc: BaseException) → bool:
    if isinstance(exc, _PROPAGATE):
        return False          # questi devono SEMPRE propagare
    if isinstance(exc, Exception):
        return True
    return type(exc).__name__ == "PanicException"
```

L'idioma al punto di chiamata è `except BaseException as exc: if not is_parse_error(exc): raise`. La disciplina è esplicita e vale la pena enunciarla: l'unica **BaseException** che questo sistema si permette di assorbire è il panic nominato: interruzione da tastiera, uscita dal processo e chiusura di un generatore passano sempre.

Il timeout di parsing è reale. La versione precedente usava `with ThreadPoolExecutor(...)`: il **FutureTimeoutError** veniva sollevato correttamente, ma all'uscita dal blocco `with` lo `shutdown(wait=True)` implicito si metteva ad aspettare proprio il thread di parsing appeso. Il timeout scattava e il chiamante restava bloccato lo stesso. Oggi `_run_with_parse_timeout()` gestisce l'esecutore a mano e chiude con `shutdown(wait=False, cancel_futures=True)`, **abbandonando** il thread orfano e scrivendolo a log come errore invece di nascondere. Il test `test_parse_timeout_real.py` mette un worker appeso per 30 secondi e pretende che il chiamante torni in meno di 5.

-Demo Format Adapter (`demo_format_adapter.py`)

Livello di resilienza per la gestione di versioni diverse del formato demo CS2.

Costanti di validazione:

Costante	Valore	Descrizione
<code>DEMO_MAGIC_V2</code>	<code>b"PBDEMS2\x00"</code>	Magic bytes CS2 (Source 2 Protobuf)
<code>DEMO_MAGIC_LEGACY</code>	<code>b"HL2DEMO\x00"</code>	Magic bytes CS:GO legacy (non supportato)
<code>MIN_DEMO_SIZE</code>	10×1024^2 (10 MB)	DS-12: demo CS2 reali sono 50+ MB, file più piccoli sono certamente corrotti o incompleti
<code>MAX_DEMO_SIZE</code>	5×1024^3 (5 GB)	Cap di sicurezza

Dataclass:

- `FormatVersion(name, magic, description, supported)` — specifica una versione nota del formato

- `ProtoChange(date, description, affected_events, migration_notes)` — record di un cambiamento protobuf noto

FORMAT_VERSIONS: Dizionario con due formati conosciuti (`cs2_protobuf` supportato, `csgo_legacy` non supportato).

PROTO_CHANGELOG: Lista cronologica dei cambiamenti noti al formato protobuf CS2 (per resilienza ai futuri aggiornamenti).

DemoFormatAdapter.validate_demo(path): Validazione a 3 fasi: (1) esistenza e dimensioni entro bounds, (2) lettura magic bytes per identificazione formato, (3) verifica supporto del formato rilevato.

-Event Registry (`event_registry.py`)

Registro canonico di **tutti gli eventi di gioco CS2** derivato dai dump SteamDatabase.

GameEventSpec dataclass con 7 campi: `name`, `category` (round/combat/utility/economy/movement/meta), `fields` (dict campo→tipo), `priority` (critical/standard/optional), `implemented` (bool), `handler_path` (opzionale), `notes`.

Categorie di eventi registrati:

Categoria	Eventi	Priorità Critica	Implementati
Round	<code>round_end</code> , <code>round_start</code> , <code>round_freeze_end</code> , <code>round_mvp</code> , <code>begin_new_match</code>	<code>round_end</code>	1/5
Combat	<code>player_death</code> , <code>player_hurt</code> , <code>player_blind</code> , etc.	<code>player_death</code>	parziale
Utility	<code>flashbang_detonate</code> , <code>hegrenade_detonate</code> , <code>smokegrenade_expired</code> , etc.	—	parziale
Economy	<code>item_purchase</code> , <code>bomb_planted</code> , <code>bomb_defused</code> , etc.	<code>bomb_planted/defused</code>	parziale
Movement	<code>player_footstep</code> , <code>player_jump</code> , etc.	—	no
Meta	<code>player_connect</code> , <code>player_disconnect</code> , etc.	—	no

Funzioni di utility: `get_implemented_events()` → lista eventi implementati. `get_coverage_report()` → rapporto copertura per categoria.

Nota (F6-33): I `handler_path` non sono validati a runtime — se i moduli gestori vengono spostati, i riferimenti diventano silenziosamente obsoleti. Aggiungere validazione `hasattr/callable` al dispatch degli eventi se l'affidabilità è critica.

-Trade Kill Detector (`trade_kill_detector.py`)

Identifica i **trade kill** — uccisioni di ritorsione entro una finestra temporale — dalle sequenze di morte nel demo.

Costante: `TRADE_WINDOW_S = 3.0` secondi. La finestra in tick è **tick-rate aware**: `int(TRADE_WINDOW_S × tick_rate)` — 192 tick a 64 tick/s, 384 tick a 128 tick/s. Il tick rate è letto dall'header della demo con fallback a `DEFAULT_TICK_RATE`.

TradeKillResult dataclass:

- `total_kills`, `trade_kills`, `players_traded`, `trade_details`
- Proprietà calcolate: `trade_kill_ratio`, `was_traded_ratio`

Algoritmo (derivato da cstat-main): Per ogni uccisione K al tick T: guarda indietro nel tempo per uccisioni effettuate dalla vittima. Se la vittima ha ucciso un compagno di squadra dell'uccisore di K entro `TRADE_WINDOW_TICKS`, segna K come trade kill e la vittima originale come "was traded". **Vincolo same-round:** Le uccisioni candidate per il trade devono appartenere allo **stesso round** (`round_num` identico). I trade cross-round non vengono conteggiati — questa è una distinzione tattica importante perché un trade ha significato strategico solo all'interno dello stesso round, dove influenza direttamente l'economia numerica dello scontro.

build_team_roster(parser): Costruisce mappatura `player_name → team_num` dai tick iniziali della partita (usa il 10° percentile dei tick per stabilità dell'assegnazione).

get_round_boundaries(parser): Estrae i tick di confine tra round dall'evento `round_end`.

-Steam API (`steam_api.py`)

Client per la Steam Web API con retry e backoff esponenziale.

Costanti: `MAX_RETRIES = 3`, `BACKOFF_DELAYS = [1, 2, 4]` secondi, tetto totale `max_total_timeout = 20s` (deadline monotonica).

_request_with_retry(...): Wrapper HTTP GET con 3 tentativi per errori di connessione/timeout. Non effettua retry su errori HTTP 4xx/5xx (li propaga al chiamante).

Funzioni principali:

- `resolve_vanity_url(vanity_url, api_key)` → risolve un URL personalizzato Steam a un SteamID a 64 bit
- `fetch_steam_profile(steam_id, api_key)` → recupera profilo giocatore (nome, avatar, ore di gioco). Auto-risolve vanity URL se l'input non è numerico

-Steam Demo Finder (`steam_demo_finder.py`)

Auto-discovery delle demo CS2 dall'installazione Steam locale.

SteamDemoFinder classe con strategia di rilevamento a 3 livelli:

Priorità	Metodo	Piattaforma
1	Registry Windows (<code>winreg</code>)	Windows
2	Percorsi comuni (generati dinamicamente per ogni drive)	Windows/Linux/macOS
3	Variabili d'ambiente	Tutte

Rilevamento drive dinamico (Windows): Usa `windll.kernel32.GetLogicalDrives()` per enumerare tutti i drive disponibili, poi cerca `Program Files (x86)/Steam`, `Program Files/Steam`, `Steam` su ogni drive.

SteamNotFoundError : Eccezione specifica quando l'installazione Steam non può essere localizzata.

Nota (F6-11): La scoperta del percorso Steam è duplicata in `ingestion/steam_locator.py` (primario). Questo modulo è supplementare (scansiona directory replay). Consolidamento differito; assicurare stessa precedenza dei percorsi quando si modifica la risoluzione.

-Modulo HLTV (`backend/data_sources/hltv/`)

Il sottosistema HLTV è composto da 3 moduli specializzati (`stat_fetcher.py`, `flaresolverr_client.py`, `docker_manager.py`) che collaborano per estrarre statistiche professionistiche dal sito HLTV.org, superando le protezioni anti-scraping di Cloudflare:

HLTVStatFetcher (`stat_fetcher.py`) — Orchestratore principale dello scraping:

- **Preflight robots.txt** prima di qualsiasi crawl
- Scopre i giocatori attraverso la pagina del ranking dei team (`fetch_top_teams`), poi per ogni giocatore recupera la pagina overview e le sotto-pagine (individual, career, opponents, clutches)
- Salva su `ProPlayer` + `ProPlayerStatCard` i campi: `rating_2_0`, `kpr`, `dpr`, `adr`, `kast`, `impact`, `headshot_pct`, `maps_played`, `opening_kill_ratio`, `opening_duel_win_pct`, `clutch_win_count`, `multikill_round_pct`, `detailed_stats_json`; i team e i roster su `ProTeam`

Statistiche estratte e salvate in `ProPlayerStatCard` :

Categoria	Statistiche
Core	<code>rating_2_0</code> , <code>kpr</code> (Kill/Round), <code>dpr</code> (Death/Round), <code>adr</code> (Damage/Round)
Efficienza	<code>kast</code> (Kill/Assist/Survival/Trade %), <code>headshot_pct</code> , <code>impact</code>
Apertura	<code>opening_kill_ratio</code> , <code>opening_duel_win_pct</code>
Tratti (JSON)	Firepower (<code>kpr_win</code> , <code>adr_win</code>), Entrying (<code>traded_deaths_pct</code>), Utility (<code>flash_assists</code>)
Approfondimenti (JSON)	Clutch (1on1/1on2/1on3), Multikill (3k/4k/5k), Carriera (rating per periodo)

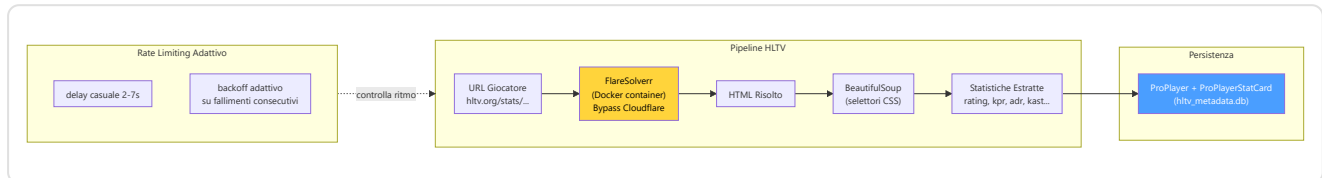
Rate limiting adattivo (in `stat_fetcher.py`): ritardo casuale tra richieste `CRAWL_DELAY_MIN_SECONDS = 2` – `CRAWL_DELAY_MAX_SECONDS = 7` secondi, con **backoff adattivo** che cresce con `_consecutive_failures` — dopo fallimenti consecutivi il fetcher rallenta progressivamente anziché insistere.

DockerManager (`docker_manager.py`) — Gestione container FlareSolvrr con strategia di avvio a cascata:

1. **Fast path:** Ritorna `True` se già in salute
2. **Docker start:** Tenta `docker start flaresolvrr`
3. **Docker Compose fallback:** Tenta `docker compose up -d`

4. **Health polling:** Verifica disponibilità ogni 3s (`_POLL_INTERVAL_S`) per max 45s (`_MAX_WAIT_S`)

FlareSolvrrClient (`flaresolvrr_client.py`) — Bypass automatico di Cloudflare JavaScript challenges. Tutte le richieste HTTP sono instradate attraverso FlareSolvrr su `http://localhost:8191/v1` (immagine Docker pinnata `flaresolvrr:v3.4.6`, unico servizio del `docker-compose.yml` di progetto). Retry: `_MAX_RETRIES = 3` con backoff esponenziale base 5 (5/15/45s), timeout 60s per richiesta. L'HTML risolto viene passato a BeautifulSoup per il parsing.



Nota architetturale: `data_sources/hltv/` è l'unico sottosistema di scraping HLTV: non esiste (più) alcun modulo `HLTVApiService`, `CircuitBreaker` o `BrowserManager` — la resilienza è affidata al backoff adattivo del fetcher e ai retry del client FlareSolvrr. Il daemon di sincronizzazione (`hltv_sync_service.py`, processo separato) è documentato in Parte 3.

Selezione automatica del pro di riferimento: Il `HybridCoachingEngine` utilizza i dati di `hltv_metadata.db` (tabelle `ProPlayer`, `ProTeam`, `ProPlayerStatCard` + le tabelle estese `ProEvent`, `ProTournament`, `ProHead2Head`, `ProMapRecord`): quando genera un'analisi, trova automaticamente il giocatore pro il cui `rating_2_0` è più vicino a quello dell'utente e lo nomina nei feedback ("la tua ADR è inferiore a quella di [nome pro]"), via `_find_best_match_pro()` in `coaching_service.py` e `_get_pro_name()` in `hybrid_engine.py`.

`hltv_scraper.py` (entry point in `data_sources/`):

- `run_hltv_sync_cycle(limit=50)` — Orchestratore del ciclo di sincronizzazione statistiche pro

-FACEIT API e Integrazione (`faceit_api.py`, `faceit_integration.py`)

faceit_api.py: Funzione singola `fetch_faceit_data(nickname)` che recupera Elo e Level FACEIT per un dato nickname. Richiede `FACEIT_API_KEY` dalla configurazione. Restituisce `{faceit_id, faceit_elo, faceit_level}` o dizionario vuoto in caso di errore.

faceit_integration.py: Client FACEIT completo con rate limiting:

Parametro	Valore	Descrizione
<code>BASE_URL</code>	<code>https://open.faceit.com/data/v4</code>	Endpoint API FACEIT v4
<code>RATE_LIMIT_DELAY</code>	6 secondi	10 req/min = 1 req ogni 6s (tier gratuito)

FACEITIntegration classe con:

- `_rate_limited_request(endpoint, params)` — richieste con rate limiting automatico; su HTTP 429 rispetta l'header `Retry-After` (cap 300s, default 60s) con `MAX_429_RETRIES = 3`
- Gestione match history, dettagli partita e download demo
- Eccezione dedicata `FACEITAPIError`

-TensorFactory — Fabbrica dei Tensori (`backend/processing/tensor_factory.py`)

La **TensorFactory** è il **sistema percettivo** del RAP Coach: converte lo stato di gioco grezzo in 3 tensori-immagine che il modello neurale può "vedere". Ogni tensore è un'immagine a 3 canali che codifica una diversa dimensione della situazione tattica: **mappa** (dove sono tutti), **vista** (cosa può vedere il giocatore) e **movimento** (come si sta muovendo).

Configurazioni:

Parametro	<code>TensorConfig</code> (Inferenza)	<code>TrainingTensorConfig</code> (Addestramento)
<code>map_resolution</code>	128 × 128	64 × 64
<code>view_resolution</code>	224 × 224	64 × 64
<code>sigma</code> (blur gaussiano)	3.0	3.0
<code>fov_degrees</code>	90°	90°
<code>view_distance</code>	2000.0 unità mondo	2000.0 unità mondo

Nota (F2-02): `TrainingTensorConfig` riduce la risoluzione da 128/224 a 64/64, ottenendo un **risparmio di memoria di ~12×**. Il contratto `AdaptiveAvgPool2d` nella `RAPPerception` produce 128-dim indipendentemente dalla risoluzione di input, ma questa garanzia è implicita — un'asserzione a runtime è raccomandata.

Costanti di rasterizzazione:

Costante	Valore	Scopo
OWN_POSITION_INTENSITY	1.5	Luminosità marcatore posizione propria
ENTITY_TEAMMATE_DIMMING	0.7	Compagni renderizzati più scuri dei nemici
ENTITY_MIN_INTENSITY	0.2	Intensità minima entità visibile
ENEMY_MIN_INTENSITY	0.3	Intensità minima nemico visibile
BOMB_MARKER_RADIUS	50.0	Raggio cerchio bomba (unità mondo)
BOMB_MARKER_INTENSITY	0.8	Opacità cerchio bomba
TRAJECTORY_WINDOW	32 tick	Finestra traiettoria (~0.5s a 64 Hz)
VELOCITY_FALLOFF_RADIUS	20.0	Celle griglia per sfumatura radiale velocità
MAX_SPEED_UNITS_PER_TICK	4.0	Velocità massima CS2 (64 tick/s)
MAX_YAW_DELTA_DEG	45.0	Soglia flick per rilevamento mira

I 3 Rasterizzatori:

1. Rasterizzatore Mappa — `generate_map_tensor(ticks, map_name, knowledge)` → `Tensor(3, res, res)`

Canale	Modalità Player-POV (con PlayerKnowledge)	Modalità Legacy (senza knowledge)
Ch0	Compagni (sempre noti) + posizione propria (intensità 1.5)	Posizioni nemici
Ch1	Nemici visibili (piena intensità) + ultimi nemici noti (decadimento esponenziale)	Posizioni compagni
Ch2	Zone utilità (fumo/molotov) + overlay bomba	Posizione giocatore

2. Rasterizzatore Vista — `generate_view_tensor(ticks, map_name, knowledge)` → `Tensor(3, res, res)`

Canale	Modalità Player-POV	Modalità Legacy
Ch0	Maschera FOV (cono geometrico 90° dalla direzione di sguardo)	Maschera FOV
Ch1	Entità visibili: compagni (dimmed ×0.7) + nemici visibili (intensità pesata per distanza)	Zona pericolo (aree NON coperte da FOV accumulato, capped a 8 tick)
Ch2	Zone utilità attive (cerchi fumo/molotov in unità mondo)	Zona sicura (recentemente visibile ma non in FOV corrente)

3. Codificatore Movimento — `generate_motion_tensor(ticks, map_name)` → `Tensor(3, res, res)`

Canale	Contenuto
Ch0	Traiettoria ultimi 32 tick — intensità $\propto$ recenza (più nuovo = 1.0, più vecchio $\rightarrow$ 0)
Ch1	Campo velocità — gradiente radiale dal giocatore, modulato dalla velocità corrente [0, 1]
Ch2	Movimento mirino — magnitudine delta yaw come blob gaussiano sulla posizione giocatore

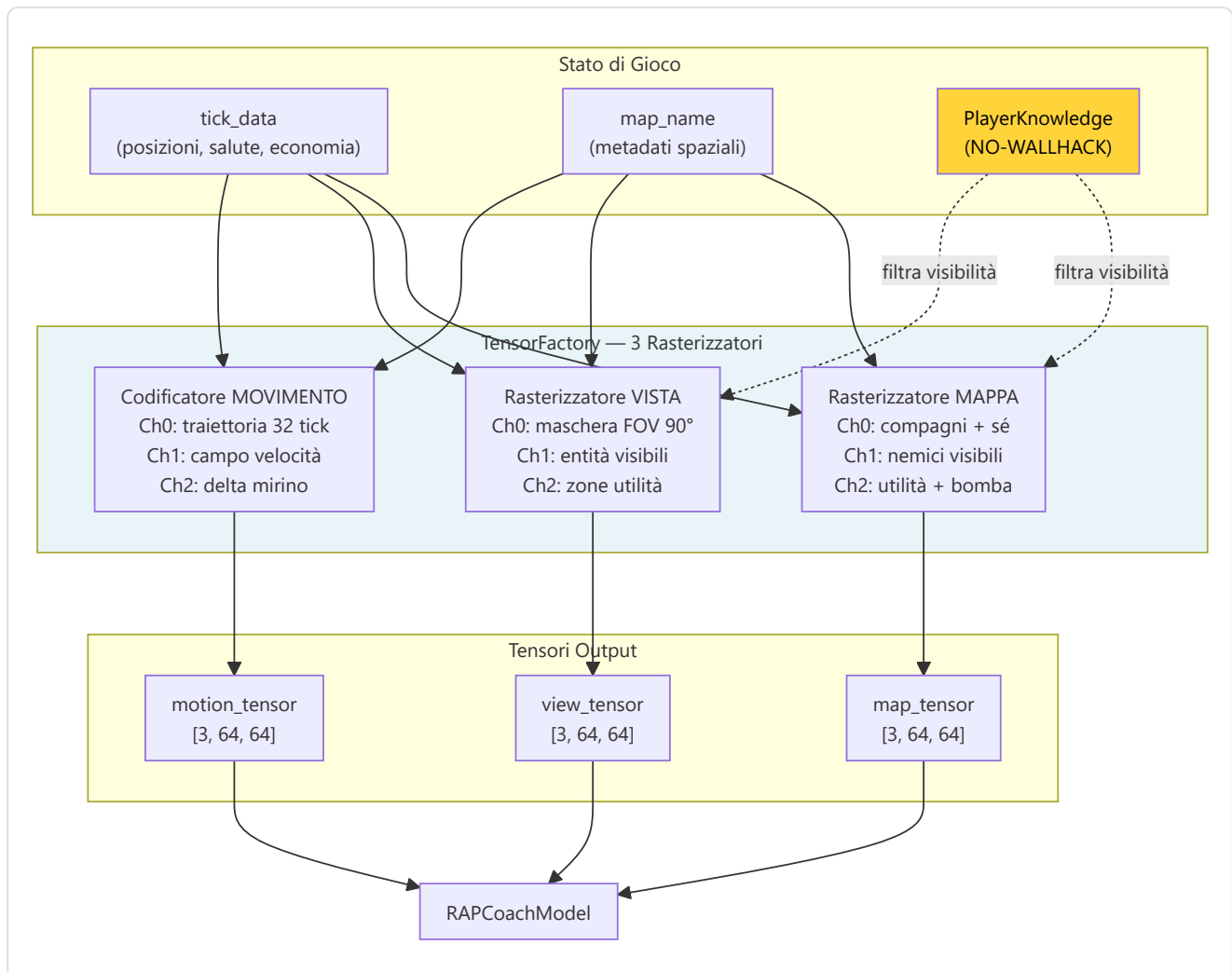
Nota (F2-03): Le demo a 128 tick/s comprimono la velocità nella metà inferiore dell'intervallo [0, 1]; normalizzazione consapevole del tick-rate in attesa di implementazione.

Integrazione NO-WALLHACK: Quando `PlayerKnowledge` è fornita, i rasterizzatori mappa e vista codificano **solo lo stato visibile al giocatore**. Le posizioni nemiche dell'ultimo avvistamento decadono esponenzialmente nel tempo. Le zone utilità sono visibili solo se nel FOV o note dal radar. Quando `knowledge=None`, il sistema degrada alla modalità legacy per retrocompatibilità.

Metodi helper:

- `_world_to_grid(x, y, meta, resolution)` — Conversione coordinate mondo $\rightarrow$ griglia. **Nota C-03:** Singolo Y-flip (`meta.pos_y - y`) per evitare doppia inversione
- `_normalize(arr)` — Normalizzazione a [0, 1]. **Nota M-10:** `arr / max(max_val, 1.0)` per prevenire amplificazione del rumore in canali sparsi
- `_generate_fov_mask(player_x, player_y, yaw, meta, resolution)` — Maschera conica 90° dalla direzione di sguardo, limitata per distanza (approssimazione 2D top-down)

Accesso Singleton: `get_tensor_factory()` — double-checked locking, thread-safe.



-Indice Vettoriale FAISS (`backend/knowledge/vector_index.py`)

Il **VectorIndexManager** fornisce ricerca semantica ad alta velocità per il sistema di conoscenza RAG (Retrieval-Augmented Generation) del coach. Utilizza FAISS (Facebook AI Similarity Search) con `IndexFlatIP` su vettori L2-normalizzati, ottenendo efficacemente una **ricerca per similarità coseno** in tempo sub-lineare.

Indici Duali:

Indice	Sorgente DB	Contenuto
"knowledge"	Tabella <code>TacticalKnowledge</code>	Embedding conoscenza tattica (strategie, posizioni, utilità)
"experience"	Tabella <code>CoachingExperience</code>	Embedding esperienze di coaching (feedback, correzioni, consigli)

Tipo di indice: `faiss.IndexFlatIP` (Inner Product) su vettori L2-normalizzati. Poiché $\cos(a, b) = \frac{a \cdot b}{(\|a\| \times \|b\|)}$, normalizzando i vettori a norma unitaria, il prodotto interno **equivale esattamente** alla similarità coseno. Intervallo risultante: $[0, 1]$ dove 1 = identico.

API pubblica:

Metodo	Descrizione
<code>search(index_name, query_vec, k)</code>	Ricerca i k vettori più simili. Lazy rebuild se dirty. Ritorna <code>List[(db_id, similarity)]</code>
<code>rebuild_from_db(index_name)</code>	Ricostruzione completa dell'indice dalla tabella DB. Thread-safe. Ritorna conteggio vettori
<code>mark_dirty(index_name)</code>	Marca l'indice per ricostruzione lazy (al prossimo <code>search()</code>)
<code>index_size(index_name)</code>	Ritorna <code>index.ntotal</code> o 0 se non costruito

Persistenza su disco:

- Formato: `{persist_dir}/{index_name}.faiss` + `{index_name}_ids.npy`
- Salvataggio: `faiss.write_index()` + `np.save()`
- Caricamento: automatico in `__init__` via `faiss.read_index()` + `np.load()`
- Directory default: `~/cs2analyzer/indexes/`

Thread-safety: Un singolo `threading.Lock()` protegge tutte le operazioni di lettura/scrittura sugli indici, i flag dirty e le operazioni di rebuild. FAISS `IndexFlatIP` è thread-safe per letture concorrenti.

Ricostruzione lazy (`mark_dirty()`): Quando nuovi dati vengono inseriti nelle tabelle Knowledge o Experience, l'indice viene marcato come "dirty" anziché ricostruito immediatamente. La ricostruzione avviene solo al prossimo `search()`, evitando rebuild multipli durante inserimenti batch.

Normalizzazione vettoriale:

```
norms = ||embedding||, per riga
normalized = embedding / max(norms, 1e-8)    # stabilità numerica
IndexFlatIP.add(normalized)
```

Fallback graduale: Se `faiss-cpu` non è installato, il singleton `get_vector_index_manager()` ritorna `None` e il sistema degrada automaticamente alla ricerca brute-force (più lenta ma funzionalmente equivalente). Questo permette al programma di funzionare anche su sistemi dove FAISS non è disponibile.

Over-fetching con costanti esplicite: Per gestire scenari di post-filtraggio (category, map_name, confidence, outcome), la ricerca recupera più risultati del necessario: `k × OVERFETCH_KNOWLEDGE = k × 10` per la Knowledge Base (filtro per categoria + mappa), `k × OVERFETCH_EXPERIENCE = k × 20` per l'Experience Bank (filtro per mappa + confidence + outcome + scoring composito). Il moltiplicatore 20× per le esperienze è doppio rispetto alla conoscenza perché i filtri sono più restrittivi (4 criteri vs 2), quindi serve un pool iniziale più ampio per garantire abbastanza risultati dopo il filtraggio.

-Contesto dei Round (`round_context.py`)

Il modulo **Round Context** è la **griglia temporale** del sistema di ingestione: converte i tick grezzi dei file demo in coordinate significative "round N, tempo T secondi" che ogni altro modulo può utilizzare per contestualizzare gli eventi di gioco.

Funzioni pubbliche:

Funzione	Input	Output	Complessità
<code>extract_round_context(demo_path)</code>	Percorso file <code>.dem</code>	DataFrame: <code>round_number</code> , <code>round_start_tick</code> , <code>round_end_tick</code>	O(n) parsing eventi
<code>extract_bomb_events(demo_path)</code>	Percorso file <code>.dem</code>	DataFrame: <code>tick</code> , <code>event_type</code> (planted/defused/exploded)	O(n) parsing eventi
<code>assign_round_to_ticks(df_ticks, round_context, tick_rate)</code>	DataFrame tick + confini round	DataFrame arricchito con <code>round_number</code> , <code>time_in_round</code>	O(n log m) via <code>merge_asof</code>

Costruzione dei confini di round (`extract_round_context`):

Il modulo analizza due tipi di eventi dal file demo:

- `round_freeze_end` — il tick in cui termina il freeze time e inizia l'azione (i giocatori possono muoversi)
- `round_end` — il tick in cui il round termina (vittoria/sconfitta)

Per ogni round, accoppia l'ultimo `round_freeze_end` che precede il `round_end` corrispondente. **Fallback:** se non viene trovato un evento `round_freeze_end` per un dato round (possibile in demo corrotti o partite interrotte), utilizza il `round_end` del round precedente come inizio, registrando un warning nel log.

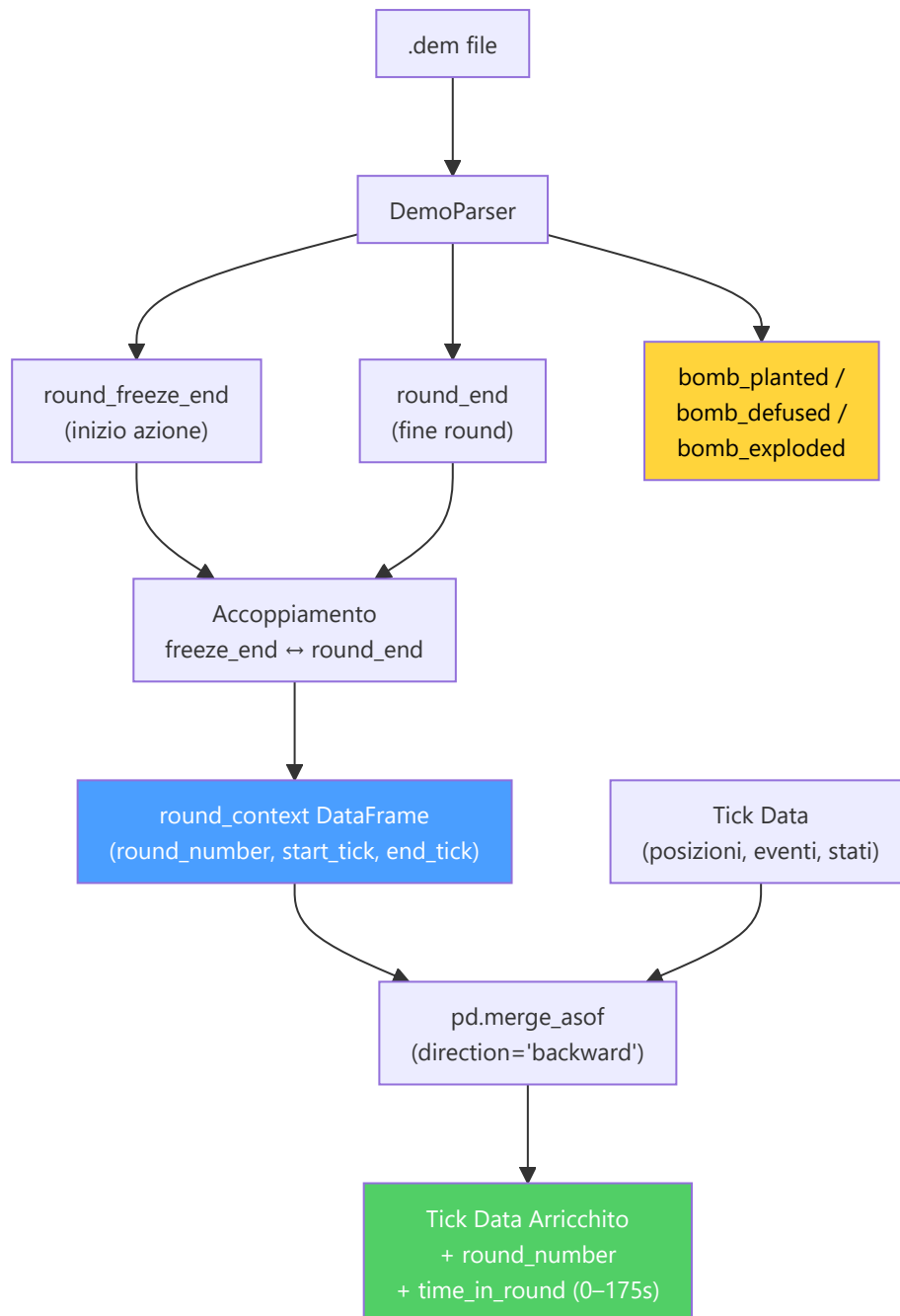
Estrazione eventi bomba (`extract_bomb_events`):

Estrae tre tipi di eventi: `bomb_planted`, `bomb_defused` e `bomb_explored`. L'aggiunta di `bomb_explored` (rimediazione H-07) permette di distinguere tra round vinti per esplosione e round vinti per eliminazione, un'informazione critica per l'analisi tattica post-plant.

Assegnazione round ai tick (`assign_round_to_ticks`):

Utilizza `pd.merge_asof` con `direction="backward"` per un'assegnazione efficiente O(n log m): per ogni tick, trova l'ultimo `round_start_tick ≤ tick`. Calcola `time_in_round = (tick - round_start_tick) / tick_rate`, limitato a [0.0, 175.0] secondi (durata massima di un round CS2). I tick prima del primo round (warmup) vengono assegnati al round 1.

Nota: L'uso di `merge_asof` al posto di un loop Python trasforma un'operazione O(n × m) in O(n log m), fondamentale per demo con milioni di tick e 30+ round.

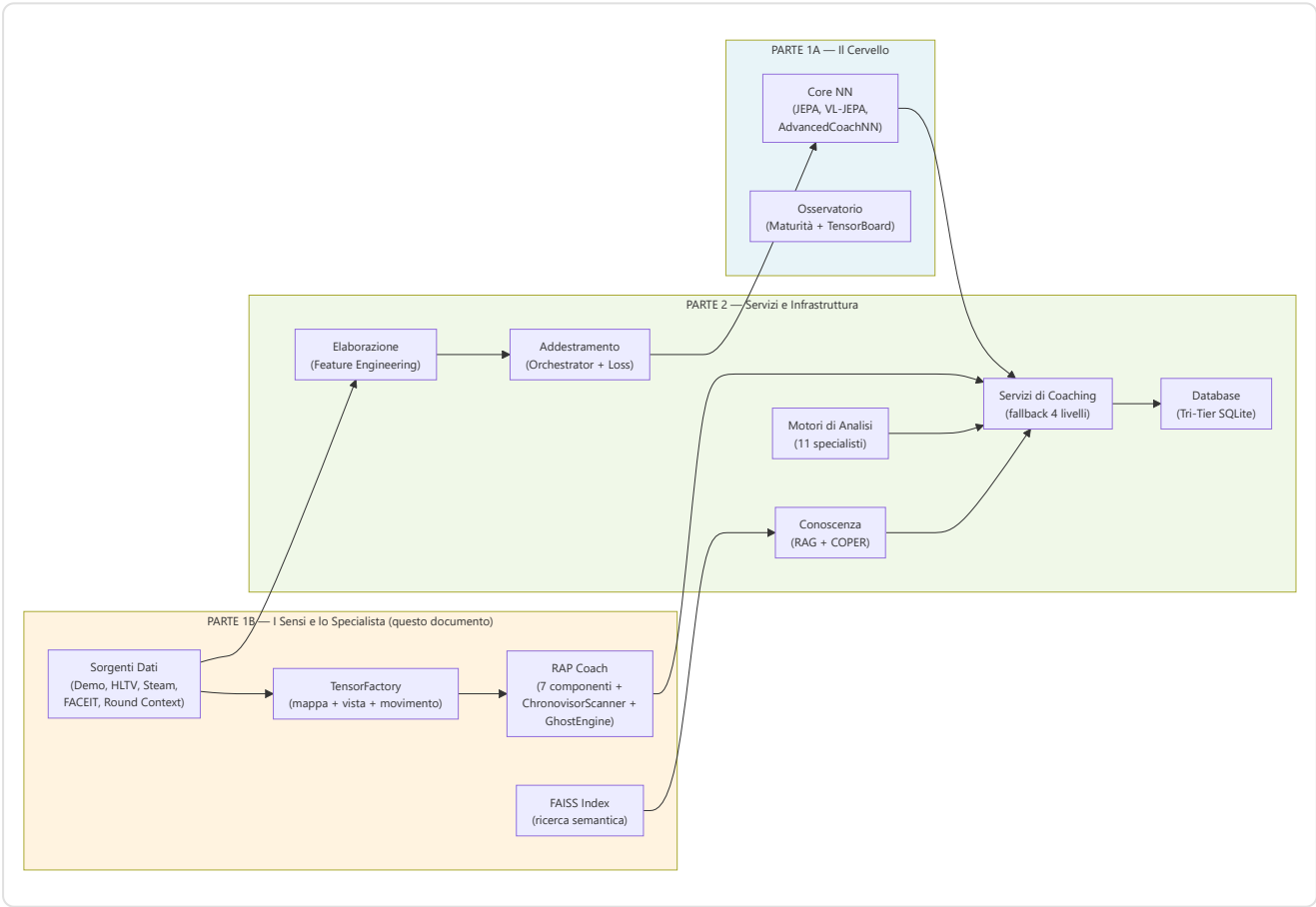


Gestione errori: Ogni fase di parsing è protetta da try/except con logging strutturato. Se il parsing fallisce completamente o non vengono trovati eventi `round_end`, la funzione restituisce un DataFrame vuoto — i moduli a valle (es. `RoundStatsBuilder`) devono gestire questo caso gracefully.

Riepilogo della Parte 1B — I Sensi e lo Specialista

La Parte 1B ha documentato i **due pilastri percettivi e diagnostici** del sistema di coaching:

Sottosistema	Ruolo	Componenti Chiave
2. RAP Coach	Il medico specialista — architettura a 7 componenti per coaching completo in condizioni POMDP	Percezione (ResNet a 3 flussi, 24 conv), Memoria (LTC 512 unità NCP con proiezione 154→256 + HopfieldLayer 4 teste/32 prototipi + ritardo attivazione NN-MEM-01 + RAPMemoryLite fallback LSTM), Strategia (4 esperti MoE Top-2 + SuperpositionLayer FiLM, context 89-dim), Pedagogia (Value Critic + Skill Adapter), Attribuzione Causale (5 categorie, segnale utilità appreso), Posizionamento (Linear 256→3), Comunicazione (template, esterna al modello), ChronovisorScanner (3 scale temporali + 50K tick safety limit + deduplicazione cross-scala + ScanResult strutturato), GhostEngine (pipeline 4-tensori con POV mode R4-04-01, hidden_state NN-40, fallback a 5 livelli)
1B. Sorgenti Dati	I sensi — acquisiscono e strutturano dati dal mondo esterno	Demo Parser (demoparser2 + rating HLTV 2.0 unificato), Demo Format Adapter (magic bytes PBDEMS2), Event Registry (schema CS2), Trade Kill Detector (finestra 3s tick-aware), Steam API (retry + backoff), Steam Demo Finder (cross-platform), HLTV (FlareSolvrr v3.4.6 + delay adattivo 2-7s + robots.txt preflight → hltv_metadata.db, 7 tabelle Pro*), FACEIT API (10 req/min, Retry-After su 429), TensorFactory (3 rasterizzatori NO-WALLHACK), FAISS (IndexFlatIP su embedding 384-dim), Round Context (merge_asof O(n log m))



Continua nella Parte 2 — Servizi di Coaching, Coaching Engines, Conoscenza e Recupero, Motori di Analisi (11), Elaborazione e Feature Engineering, Modulo di Controllo, Progresso e Tendenze, Database e Storage (Tri-Tier), Pipeline di Addestramento e Orchestrazione, Funzioni di Perdita